



IT SIMPLERA INSTITUTE

Machine Learning Internship Program

Week 4 Task Documentation

Unsupervised Learning & Customer Segmentation

K-Means Clustering | Hierarchical Clustering | Credit Card Customer Data

Dataset: Credit Card Dataset for Clustering (Kaggle)

1. Objective of the Task

This week moves away from supervised learning into unsupervised learning, where there is no target column to predict. The goal is to discover hidden patterns and natural groupings within customer credit card behavioral data using clustering algorithms.

Customer segmentation of this kind is one of the most widely used machine learning applications in banking and finance. Grouping customers into meaningful segments allows a business to:

- Target marketing campaigns more precisely toward specific customer behaviors.
- Manage credit risk by identifying high cash-advance or high-balance users.
- Personalize services and offers based on spending patterns.

Two complementary approaches were implemented and compared: K-Means clustering (Part 1) and Agglomerative Hierarchical Clustering (Part 2).

2. Dataset Overview

Property	Details
Name	Credit Card Dataset for Clustering
Source	Kaggle — arjunbhasin2013/ccdata
Records	8,950 active credit card holders
Features	18 columns covering balance, purchase frequency, credit limit, cash advance usage, and payment history
Time span	Behavioral data collected over 6 months

3. Approach & Methodology

Part 1 — K-Means Clustering

1. Load the dataset and inspect its shape, column names, and data types.
2. Drop the CUST_ID column since it is an identifier, not a behavioral feature.
3. Detect and handle missing values using median imputation.
4. Scale all features with StandardScaler so that no single feature dominates the distance calculation.
5. Run K-Means for $k = 2$ through 10, recording inertia for each value of k .
6. Plot the Elbow Curve (k vs. inertia) to visually estimate the optimal cluster count.
7. Calculate and plot the Silhouette Score for each k , and cross-check it against the elbow result.
8. Fit the final K-Means model using the chosen optimal k and assign a cluster label to every customer.
9. Profile each cluster by computing per-cluster feature means and visualize them as a heatmap.
10. Interpret each cluster in business terms (e.g., high spenders, low-balance customers, cash-advance users).

Part 2 — Hierarchical Clustering

11. Draw a random sample of 300 rows from the scaled dataset (hierarchical clustering does not scale well to the full 8,950 rows).
12. Apply Ward-linkage Agglomerative Clustering using scipy and plot a labeled Dendrogram with a horizontal threshold line marking the suggested cut.
13. Apply scikit-learn's AgglomerativeClustering using the same optimal cluster count chosen in Part 1.
14. Cross-tabulate the hierarchical cluster labels against the K-Means labels for the same sample to check agreement.
15. Write a comparison report on interpretability, scalability, and which algorithm suits production use.

4. Implementation & Results

4.1 Data Loading & Inspection

The dataset was loaded from CC GENERAL.csv, containing 8,950 rows and 18 columns: CUST_ID, BALANCE, BALANCE_FREQUENCY, PURCHASES, ONEOFF_PURCHASES, INSTALLMENTS_PURCHASES, CASH_ADVANCE, PURCHASES_FREQUENCY, ONEOFF_PURCHASES_FREQUENCY, PURCHASES_INSTALLMENTS_FREQUENCY, CASH_ADVANCE_FREQUENCY, CASH_ADVANCE_TRX, PURCHASES_TRX, CREDIT_LIMIT, PAYMENTS, MINIMUM_PAYMENTS, PRC_FULL_PAYMENT, and TENURE.

```
Loaded file: c:\Users\Kiran\Desktop\ITSimplera_Institute\Week_04\data\CC_GENERAL.csv
Shape: (8950, 18)
Columns: ['CUST_ID', 'BALANCE', 'BALANCE_FREQUENCY', 'PURCHASES', 'ONEOFF_PURCHASES',
Data types:
CUST_ID          str
BALANCE          float64
BALANCE_FREQUENCY float64
PURCHASES        float64
ONEOFF_PURCHASES float64
INSTALLMENTS_PURCHASES float64
CASH_ADVANCE     float64
PURCHASES_FREQUENCY float64
ONEOFF_PURCHASES_FREQUENCY float64
PURCHASES_INSTALLMENTS_FREQUENCY float64
CASH_ADVANCE_FREQUENCY float64
CASH_ADVANCE_TRX int64
PURCHASES_TRX    int64
CREDIT_LIMIT     float64
PAYMENTS         float64
MINIMUM_PAYMENTS float64
PRC_FULL_PAYMENT float64
TENURE           int64
dtype: object
```

	CUST_ID	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES
0	C10001	40.900749	0.818182	95.40	0.00	
1	C10002	3202.467416	0.909091	0.00	0.00	
2	C10003	2495.148862	1.000000	773.17	773.17	
3	C10004	1666.670542	0.636364	1499.00	1499.00	
4	C10005	817.714335	1.000000	16.00	16.00	

4.2 Dropping the Identifier Column

The CUST_ID column was identified and dropped, since it uniquely identifies a customer and carries no behavioral meaning for clustering. After removal, 17 numeric features remained.

4.3 Handling Missing Values

Missing values were found in two columns: CREDIT_LIMIT (1 missing) and MINIMUM_PAYMENTS (313 missing). Both were filled using median imputation, because the median is more robust than the mean and is not distorted by outliers, which are common in financial spending data.

Column	Missing Values (Before)	Missing Values (After)
CREDIT_LIMIT	1	0
MINIMUM_PAYMENTS	313	0

```
Missing values before cleaning:
CREDIT_LIMIT      1
MINIMUM_PAYMENTS  313
dtype: int64

Missing values after cleaning:
Series([], dtype: int64)
```

4.4 Feature Scaling

All 17 remaining features were standardized using StandardScaler. Scaling is mandatory before clustering because algorithms such as K-Means rely on Euclidean distance between points. Features like BALANCE or CASH_ADVANCE naturally have much larger numeric ranges than frequency-based features (which range between 0 and 1); without scaling, those large-range features would dominate the distance calculation and bias the clusters toward them.

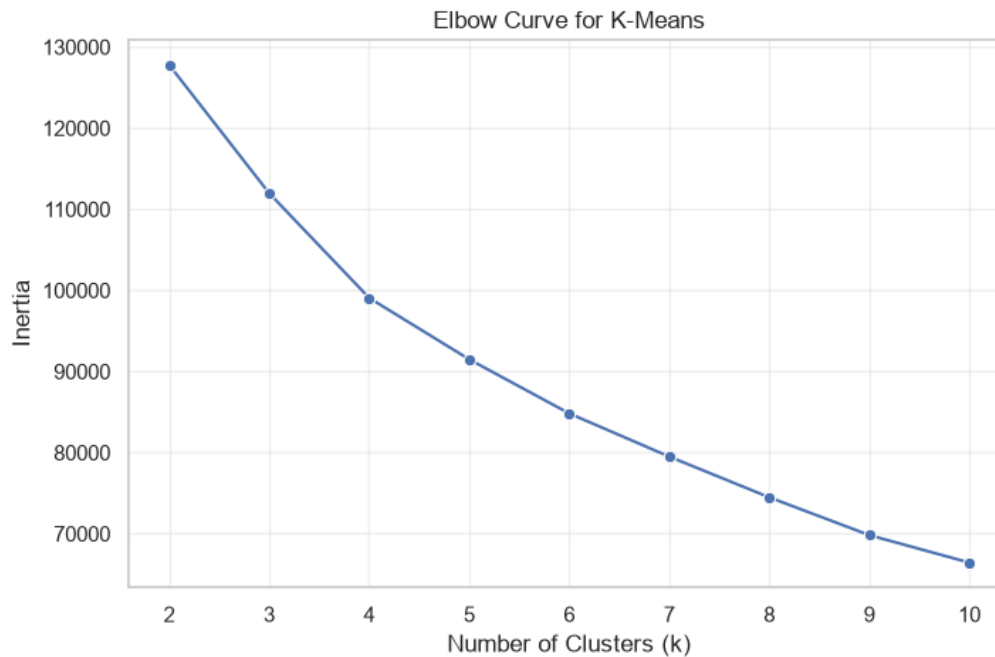
	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES
0	-0.731989	-0.249434	-0.424900	-0.356934	-0.349079
1	0.786961	0.134325	-0.469552	-0.356934	-0.454576
2	0.447135	0.518084	-0.107668	0.108889	-0.454576
3	0.049099	-1.016953	0.232058	0.546189	-0.454576
4	-0.358775	0.518084	-0.462063	-0.347294	-0.454576

4.5 K-Means: Elbow Method

K-Means was run for k = 2 through 10 and inertia (within-cluster sum of squares) was recorded for each value:

k	Inertia
2	127,784.53
3	111,973.97
4	99,061.94

k	Inertia
5	91,490.50
6	84,826.59
7	79,506.96
8	74,484.88
9	69,828.70
10	66,442.18



4.6 K-Means: Silhouette Score

The Silhouette Score was computed for the same range of k values to validate the elbow result. The highest silhouette score was obtained at k = 3, which was selected as the optimal number of clusters. This is consistent with the elbow curve, where the rate of decrease in inertia visibly slows down around k = 3–4.

[ATTACH SCREENSHOT HERE: Silhouette Score plot (k vs. silhouette score)]

4.7 Final K-Means Model (k = 3)

The final K-Means model was fit with k = 3 and cluster labels were assigned to all 8,950 customers. The resulting cluster sizes were:

Cluster	Number of Customers
0	6,119
1	1,235

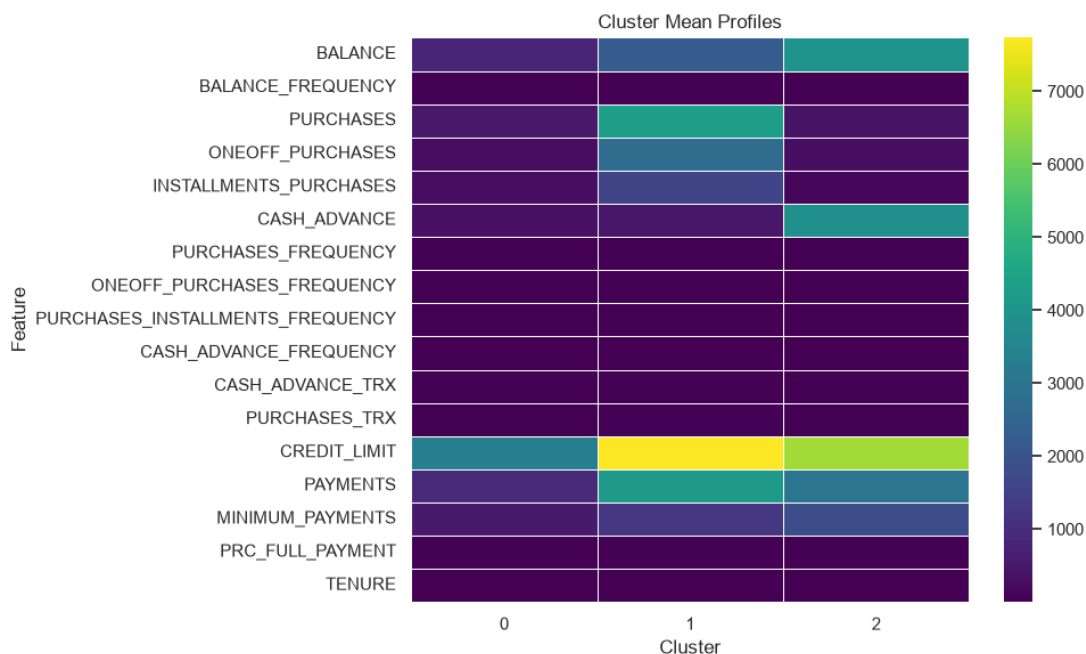
Cluster	Number of Customers
2	1,596

4.8 Cluster Profiling & Heatmap

The mean value of every feature was computed per cluster to build a cluster profile table, a sample of which (first 5 features) is shown below:

Feature	Cluster 0	Cluster 1	Cluster 2
BALANCE	799.75	2,220.00	3,989.14
BALANCE_FREQUENCY	0.84	0.98	0.96
PURCHASES	505.53	4,268.52	384.53
ONEOFF_PURCHASES	253.12	2,717.83	248.72
INSTALLMENTS_PURCHASES	252.73	1,551.18	135.89

These means were visualized as a heatmap across all 17 features to make the differences between clusters easy to compare at a glance.



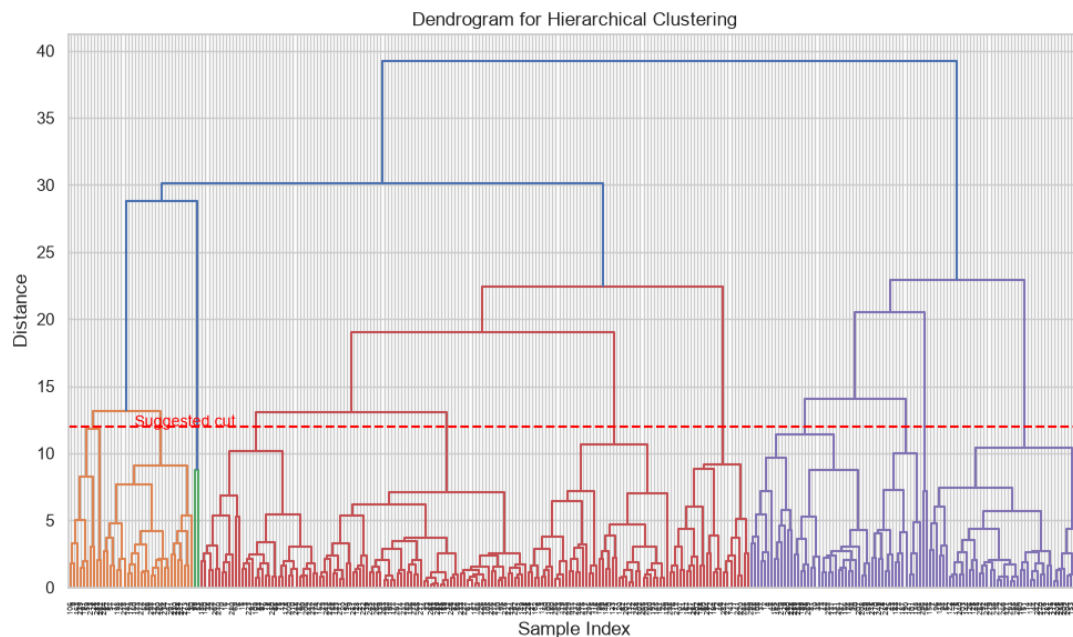
4.9 Cluster Interpretation (Business Terms)

- **Cluster 0 — Low-balance, moderate spenders:** the largest group (6,119 customers) with the lowest average balance and moderate purchase activity. Likely everyday, budget-conscious cardholders.
- **Cluster 1 — High spenders:** smaller group (1,235 customers) with the highest purchases, one-off purchases, and installment purchases. These are high-value customers ideal for premium offers and loyalty rewards.

- **Cluster 2 — High-balance / cash-advance-leaning customers: 1,596 customers carrying the highest average balance but relatively low purchase volume, suggesting revolving balances or cash-advance usage — a segment relevant for credit risk monitoring.**

4.10 Hierarchical Clustering — Dendrogram

A random sample of 300 rows was drawn from the scaled dataset. Ward-linkage Agglomerative Clustering was applied using scipy, and a dendrogram was plotted with a horizontal red threshold line at a distance of 12 to mark the suggested cut point.



4.11 Agglomerative Clustering (scikit-learn)

AgglomerativeClustering was then applied to the same 300-row sample using the same optimal cluster count ($k = 3$) chosen in Part 1. The resulting cluster sizes were:

Hierarchical Cluster	Number of Customers (sample of 300)
0	39
1	98
2	163

4.12 Comparing K-Means and Hierarchical Clustering

A cross-tabulation was built between the hierarchical cluster labels and the K-Means labels for the same 300-row sample:

Hierarchical \ K-Means	Cluster 0	Cluster 1	Cluster 2
0	0	1	38
1	55	42	1

Hierarchical \ K-Means	Cluster 0	Cluster 1	Cluster 2
2	151	1	11

The cross-tabulation shows reasonable agreement between the two methods: hierarchical cluster 2 aligns closely with K-Means cluster 0 (151 of 163 overlap), and hierarchical cluster 0 aligns closely with K-Means cluster 2 (38 of 39 overlap). Hierarchical cluster 1 is more mixed between K-Means clusters 0 and 1, showing partial disagreement at the boundary between the low-balance and high-spender groups.

4.13 Comparison Report

Both algorithms discovered broadly similar customer segments, confirming that the underlying structure in the data is genuine rather than an artifact of one algorithm. In practice:

- K-Means is faster, easier to scale to the full dataset (8,950+ rows), and simple to re-run as new customer data arrives — making it the more practical choice for production.
- Hierarchical Clustering provides a more interpretable tree-like structure (the dendrogram) that is useful for exploratory analysis and for visually understanding how clusters merge, but it does not scale well to large datasets.
- Recommendation: K-Means is recommended for the real-world business use case because of its efficiency and ease of maintenance, while hierarchical clustering is a valuable exploratory/validation step for confirming the number and structure of clusters.

6. Conclusion

This task provided hands-on experience with unsupervised learning for customer segmentation. Preprocessing (missing-value handling and feature scaling), K-Means clustering with elbow and silhouette validation, cluster profiling, and hierarchical clustering with dendrogram-based validation were all implemented end-to-end on real credit card behavioral data. The resulting three customer segments — low-balance moderate spenders, high spenders, and high-balance/cash-advance-leaning customers — offer directly actionable insight for marketing targeting, risk management, and personalized service in a banking context.